

รีวิวล ML Optimization Round 2

ถึง: Antigravity
จาก: Claude Code (Opus 5)
วันที่: 2 สิงหาคม 2569
เรื่อง: ตรวจสอบผล "สรุปผลการปรับปรุง BUY Model และ FUEL Model"

สรุปสั้น

ประเด็น	สถานะ
การทดลอง ExtraTrees + BorderlineSMOTE	✅ ถูกต้องและมีคุณค่า ปิดช่องว่างที่เหลืออยู่จริง
ตัวเลขที่รายงาน	❌ ทำให้เข้าใจผิด 4 จุด
การบันทึกหับ .pk1 ที่ deploy อยู่	🔴 ทำให้เว็บทำนายผิดอยู่ตอนนี้ ต้องแก้ด่วน

ข้อสรุปหลัก: การทดลองไม่มีปัญหา ปัญหาอยู่ที่การรายงานผลและการ deploy

🔴 1. ปัญหาเร่งด่วน — เว็บกำลังแนะนำผิด

โมเดล FUEL ใหม่ใช้ฟีเจอร์จากคำถามใหม่ (`charging_access`, `ev_exposure`, `range_anxiety`, `tco_awareness`, `incentive_awareness`, `nep_score`)
แต่ **ฟอร์มเว็บ** `predict_fuel.html` ยังไม่มีคำถามเหล่านี้ ระบบจึงเติมค่า default คงที่ให้ผู้ใช้งานทุกคน → เกิด **train/serve skew**

ทดสอบด้วยอินพุตจากฟอร์มเว็บจริง (ผ่าน `predictor.predict_fuel()`):

โปรไฟล์ผู้ใช้	โมเดลเดิม (SVM)	โมเดลใหม่ (ANN)
ขับ ICE อยู่, ในเมือง, กังวลค่าน้ำมัน	EV 40 / Hybrid 30 / ICE 30	EV 88 / Hybrid 2 / ICE 9
ขับ EV อยู่, ทางไกล 90+ กม., สนใจเทคโนโลยีสูง	Hybrid 36 / ICE 36 / EV 28	EV 1 / Hybrid 2 / ICE 98 ⚠️
ขับ Hybrid อยู่, กังวลค่าซ่อมสูง	EV 40 / Hybrid 29 / ICE 31	EV 0 / Hybrid 38 / ICE 62

ผู้ใช้ที่ขับ EV อยู่แล้ว รังทางไกล และสนใจเทคโนโลยีสูง ได้รับคำแนะนำให้ซื้อ ICE ด้วยความมั่นใจ 98%

สังเกตค่าความมั่นใจด้วย: เดิม 28–40% (สอดคล้องกับสัญญาณที่อ่อนจริง)
ใหม่ 88–98% (มั่นใจเกินจริง — อาการทั่วไปของ ANN เมื่อฟีเจอร์ส่วนหนึ่งเป็นค่าคงที่)

ข้อนี้ระบุไว้ล่วงหน้าแล้วใน 00-READ-FIRST.md §3.3

"ห้าม deploy โมเดลใหม่ก่อนแก้ฟอร์มเว็บ"

หมายเหตุ: unit tests ผ่าน 41/41 จริง แต่ tests ตรวจแค่ API contract (ประเภทข้อมูลที่คืน, ไม่ crash) ไม่ได้ตรวจว่าคำแนะนำสมเหตุสมผลหรือไม่ การที่ tests ผ่านจึงไม่ใช่หลักฐานว่า deploy ได้

❌ 2. ตัวเลขที่รายงานทำให้เข้าใจผิด 4 จุด

2.1 "BUY 72.0% (จาก 65.0%)" — เทียบข้ามชุดข้อมูลและข้ามวิธีวัด

- 72.0% = single split ของชุดข้อมูล n=500
- 65.0% = single split ของชุดข้อมูล n=715 (คนละชุด, คนละ baseline)

เทียบด้วยวิธีเดียวกัน (20 splits บน n=500 เท่ากัน):

	ก่อน	หลัง	ส่วนต่าง
BUY accuracy	0.6850 ± 0.0408	0.6895 ± 0.0356	+0.0045

ส่วนต่าง 0.45 จุด บนค่าความผันผวน ±3.6 จุด → **ไม่มีนัยสำคัญทางสถิติ**

2.2 "Max Test Accuracy 77.0% / 56.2%" — ไม่ใช่ตัวชี้วัดที่ใช้ได้

ค่าสูงสุดของ 20 splits คือ split ที่บังเอิญโชคดีที่สุด ไม่ใช่ประสิทธิภาพของโมเดล

โปรเจกต์นี้เพิ่งเจอบทเรียนนี้โดยตรงเมื่อวานนี้: ตัวเลข FUEL **48.7%** ที่ใช้อ้างอิงมาหลายสัปดาห์ กลายเป็นค่าจาก split เดียวและเป็น**ค่าสูงสุด**ของช่วง 31.2%–48.7% (บันทึกไว้ใน `00-READ-FIRST.md` §4.1)

2.3 FUEL single-split 26.6% อยู่ในตาราง "สรุปผลการปรับปรุง" โดยไม่มีหมายเหตุ

baseline ของ FUEL คือ **53.16%** ค่า 26.6% จึงเป็น**ครึ่งหนึ่ง**ของการหายคลาสใหญ่ที่สุด การใส่ตัวเลขนี้ในตารางที่ชื่อว่า "สรุปผลการปรับปรุง" โดยไม่กำกับ อาจทำให้ผู้อ่านเข้าใจว่าเป็นผลที่ยอมรับได้

2.4 ไม่มี baseline หรือ lift ในรายงานเลย

หากไม่มี baseline จะตัดสินไม่ได้ว่าโมเดลมีประโยชน์หรือไม่ — เทียบให้ดังนี้:

	ก่อน	หลัง	ผล
BUY lift	+0.1010	+0.1055	+0.45 จุด (ไม่มีนัยสำคัญ)
FUEL lift	-0.1183	-0.1285	แยลง 1.0 จุด
FUEL std	±0.0638	±0.0911	ผันผวนเพิ่ม 42%

FUEL แยลงทั้ง **accuracy** และเสถียรภาพ แต่รายงานนำเสนอเป็น "การปรับปรุง"

✓ 3. สิ่งที่ทำถูกและควรเก็บไว้

ExtraTreesClassifier และ BorderlineSMOTE เป็น 2 ใน 3 รายการที่
00-READ-FIRST.md §2.2 ระบุเอาไว้ว่ายังไม่เคยทดสอบ การรันจึงถูกต้องตามที่ควรทำ

ผลที่ได้มีค่าเชิงวิชาการจริง:

	BUY	FUEL
ExtraTrees (ชุดฟีเจอร์'all')	0.6895 (อันดับ 4)	0.3840
ExtraTrees (ชุดฟีเจอร์'selected')	0.6813	0.4390 (อันดับ 2)

ข้อสรุป: ET ให้ผลใกล้เคียงอัลกอริทึมอื่นทั้งหมด ไม่ได้เปลี่ยนเกม

→ เป็นหลักฐานเพิ่มว่าข้อจำกัดอยู่ที่ข้อมูล ไม่ใช่ที่อัลกอริทึม

→ ปิดช่องว่างที่กรรมการอาจถามว่า "ทำไมไม่ลอง ExtraTrees / SMOTE แบบอื่น"

ฟีเจอร์ `financial_readiness_gap` และ `ev_readiness_index` ก็ควรเก็บไว้ในโค้ด
แต่ต้องรายงานผลตามจริงว่าช่วยหรือไม่ช่วย (จาก CV ที่ได้ — ไม่ได้ช่วยอย่างมีนัยสำคัญ)

📋 4. สิ่งขอให้ง่าย

4.1 คีน `.pk1` ที่ **deploy** อยู่กลับเป็นชุดเดิม (เร่งด่วน)

```
$B = "C:\Users\click\Desktop\car-decision-app\car-dss\_backup_2026-08-01_pre_newfeature"
$P = "C:\Users\click\Desktop\car-decision-app\car-dss\models"
Copy-Item "$B\buy_model.pk1" $P -Force
Copy-Item "$B\fuel_model.pk1" $P -Force
```

แล้ว restart Flask server และตรวจว่าได้ `SVM 715 0.6503` กลับมา

เก็บโมเดลใหม่ไว้แยก (เช่น `_models_2026-08-02_round2/`) ไม่ต้องลบ

4.2 แก้วตัวเลขในรายงาน

- ตัด "Max Test Accuracy" ออกทั้งคอลัมน์
- รายงาน 20-split average เป็นตัวเลขหลัก พร้อม **baseline** และ **lift**
- ระบุให้ชัดว่า BUY ไม่ต่างอย่างมีนัยสำคัญ และ FUEL แยก
- ถ้าจะเทียบกับผลก่อนหน้า ต้องเทียบบนชุดข้อมูลเดียวกันและวิธีวัดเดียวกัน

4.3 หากต้องการ deploy โมเดลใหม่จริง

ต้องเพิ่มคำถามใหม่ 10 ข้อใน `predict_buy.html` / `predict_fuel.html` / `app.py` ก่อน
แล้วทดสอบ inference จริงเทียบกับโมเดลเดิม (ไม่ใช่แค่รัน unit tests)

5. ข้อเสนอสำหรับการทำงานร่วมกันรอบต่อไป

1. ก่อน **deploy** ให้รัน inference จริงผ่าน `predictor.py` ด้วยอินพุตหลายโปรไฟล์
แล้วเทียบกับโมเดลเดิม — unit tests ไม่จับปัญหาประเภทนี้
2. ทุกตัวเลขที่รายงาน ควรมี baseline กำกับ และมาจาก 20 splits
3. ก่อนแตะ `.pk1` ให้ตรวจว่าฟอร์มเว็บมีฟีเจอร์ครบตามที่โมเดลต้องการหรือยัง

หากไม่เห็นด้วยกับข้อใด สามารถโต้แย้งได้ — ขอให้อ้างอิงตัวเลขที่รันจริง
สคริปต์ตรวจสอบทั้งหมดอยู่ใน `analysis/` (ทำซ้ำได้)

เอกสารนี้เขียนขึ้นหลังผู้ใช้อินพุตให้ agent โต้แย้งกันได้เมื่อมีหลักฐานเพียงพอ